

Mnemon: An Agent-Based Study Assistant

with OCR, Retrieval, and Speech

CANDIDATE

Antonis Geralis

PROJECT ADVISOR

Ioannis Katakis

DEGREE PROGRAM

Bachelor of Science in Computer Science

Department of Computer Science
School of Sciences and Engineering
University of Nicosia

Problem Context

Study is repeated transformation of fragmented course material.

STARTING POINT

Slides, scans, and notes

WHAT CHANGES

Prepare, search, and transform the material

REVISION FORMATS

Notes, practice, and audio

- Students revise from lecture slides, scans, textbook pages, and personal notes
- The same source must become notes, explanations, flashcards, quizzes, essay practice, or audio
- A plain LLM chat makes fragmented course material harder to manage and answers harder to trace
- The workflow must remain source-grounded, modular, and inspectable

Core thesis

Coordinate the workflow around the student's material. Do not treat revision as one unstructured prompt.

Problem Statement

1. Input readiness

OCR converts scanned documents into text that can be searched and reused.

2. Source retrieval

Index course material, then retrieve relevant passages for each student query.

3. Output form

Revision requires different outputs: explanation, active recall, essay practice, and listening.

Aim, Objectives, and Scope

Aim and objectives

- Use OCR, retrieval, generation, and speech for common revision tasks
- Keep OCR, RAG, and TTS as separate tools that can be run independently
- Evaluate tool reliability, OCR performance, and recorded workflows

Scope and limitations

- System-design and engineering thesis
- No claim of measured grade improvement
- No replacement for instructor feedback or student judgement
- Remote model quality, latency, cost, and privacy remain practical constraints

Study Workflow Requirements

OCR

Converts visual material into text.
OCR errors can affect every later output.

INSPECTABLE ARTIFACT

OCR page records

RAG

Retrieves relevant passages from the student's material so answers stay grounded in the source.

INSPECTABLE ARTIFACTS

retrieved passages

TTS

Rewrite diagrams, tables, and formulas for listening, then turn the text into speech.

INSPECTABLE ARTIFACT

text prepared for speech

Intermediate artifacts keep each stage visible.

Architecture: Flexible Agent, Deterministic Tools

The agent chooses the workflow. The tools perform the processing.

study-assistant

Selects one mode: transcription, notes, flashcards, quiz, essay practice, or audio.

Tool responsibilities

Handle caching, chunking, retries, output order, and clear failures.

PROCESSING TOOLS

pdfocr

Extract text from PDFs

chunkvec

Store and search material

chunktts

Create the final audio file

Engineering Decisions

Standalone command-line tools

OCR, retrieval, and speech can be run, inspected, cached, or tested independently.

Preserve source order

Concurrent requests may complete out of order, but published pages and chunks preserve source order.

Shared libraries

The tools share common code for HTTP requests and JSON handling.

Handle partial outputs

OCR keeps successful pages when some requests fail. Incomplete audio is not published.

Workflow Evaluation Method

Evaluate the saved evidence, not only the final response.

Evaluation criteria

- Source grounding against prepared material
- Fit for the requested study mode
- Inspectable intermediate outputs
- Reproducible runs
- Visible limitations rather than hidden failures

Recorded workflows

- **Essay practice:** Association Analysis slides
- **RAG revision:** Naive Bayes textbook section
- **Flashcards:** Anomaly Detection slides
- **Notes to audio:** Clustering notes

Testing and Recorded Runs

What the code controls

- pdfocr: page selection, request IDs, retry queue, JSONL output
- chunkvec: chunk parser, embeddings configuration, SQLite/vector integration
- chunktts: chunk splitting, retry handling, audio validation, final file creation
- relay, jsonx, openai: transport and parsing helpers

What remote models affect

- Use tests for behaviour controlled by the code
- Use recorded runs for behaviour that depends on remote models
- Check ordering, failures, and tool boundaries
- Record limitations alongside the results

Evaluation: OCR Throughput

72-PAGE SLIDE PDF BENCHMARK

Running OCR requests concurrently made OCR practical while preserving page order.

15.89x

speedup

up to 32 concurrent requests

72/72

pages completed

in both runs

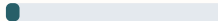
RUNTIME COMPARISON

One request at a time



316.66

Up to 32 concurrent
requests



19.93

Recorded runtime; varies with provider latency, rate limits, network conditions, and document complexity.

Evaluation: OCR Model Benchmark

68 ACADEMIC PAGES FROM 34 PDFS

Model	CER	WER	Order F1	Math F1
PaddleOCR-VL	0.463	0.473	0.375	0.725
olmOCR 2	0.468	0.489	0.325	0.674
DeepSeek-OCR	0.586	0.651	0.145	0.468

Model	Total cost
PaddleOCR-VL	\$0.0420
olmOCR 2	\$0.0214
DeepSeek-OCR	\$0.0041

Benchmark pages

Hand-labelled academic pages with equations, tables, diagrams, and multi-column layouts.

Selection tradeoff

olmOCR 2 was less likely to miss important content and cost less than the most accurate model.

Workflow Evaluation Results

Essay practice

Produced useful exam-style questions, but OCR errors make source tracing important.

Flashcards

Worked well for active recall: concise and examinable, but future checks should cover gaps, duplication, and formula clarity.

Retrieval

Strongest result: the answer matched the course material, but the retrieved passages should be shown beside it.

Study notes to audio

Produced listenable revision material, but audio cannot fully represent figures, tables, or equations.

Results and Limitations

Interpretation

- Separate tools make the workflow easier to inspect and test
- Concurrent requests reduced OCR runtime in the recorded benchmark
- Retrieval quality depends on how material is split, labelled, and retrieved

Threats and limitations

- No controlled study of learning outcomes or retention
- TTS evaluation checks the output file, not whether the audio sounds natural
- OCR results depend on the documents and provider
- Model behaviour, pricing, latency, and availability can drift

Contributions

Conceptual

Organises course material into revision formats that remain grounded in the source.

Technical

Adds ordering, retries, clear failures, and saved intermediate artifacts.

Architectural

Separates the agent from OCR, retrieval, and speech tools.

Evaluation

Provides evidence from tests, benchmarks, and recorded workflows.

Future Work

Next engineering steps

- Record each run from source PDF to final output
- Test retrieval with fixed questions and expected passages
- Test OCR on more documents and repeat the measurements
- Add checks for formulae, diagrams, duplicate cards, and coverage gaps

Further evaluation

- Compare providers and models for cost, latency, quality, and privacy
- Evaluate local or private model inference for sensitive course material
- Ask students to evaluate whether the outputs are useful
- Study effects on retention and exam preparation

Conclusion

Main conclusion

study-assistant gives an agent open tools that can be inspected, changed, and run independently.

15.89x

OCR speedup

recorded OCR benchmark

RAG

strongest workflow result

course-specific answer; retrieved passages should be shown

TTS

revision material in audio form

text is prepared before speech generation

Questions

Mnemon: An Agent-Based Study Assistant with OCR, Retrieval, and Speech

Antonis Geralis

Department of Computer Science, University of Nicosia